Listing 8.13. Testing the trained agent

```
eps=0.1
done = True
state_deque = deque(maxlen=params['frames_per_state'])
for step in range(5000):
    if done:
        env.reset()
        state1 = prepare_initial_state(env.render('rgb_array'))
    q_val_pred = Qmodel(state1)
    action = int(policy(q_val_pred,eps))
    state2, reward, done, info = env.step(action)
    state2 = prepare_multi_state(state1,state2)
    state1=state2
    env.render()
env.close()
```

There's not much to explain here if you followed the training loop; we're just extracting the part that runs the network forward and takes an action. Notice that we still use an epsilon-greedy policy with epsilon set to 0.1. Even during inference, the agent needs a little bit of randomness to keep it from getting stuck. One difference to notice is that in test (or inference) mode, we only enact the action once and not six times like we did in training. Assuming you get the same results as us, your trained agent should make fairly consistent forward progress and should be able to jump over obstacles (figure 8.20). Congratulations!

Figure 8.20. The Mario agent trained only from intrinsic rewards successfully jumping over a chasm. This demonstrates it has learned basic skills without any explicit rewards to do so. With a random policy, the agent would not even be able to move forward, let alone learn to jump over obstacles.